

LearnHub Cameroon — Frontend Software Requirements Specification

Repo: `learnhub-web` • **Version:** 1.0 • **Audience:** Frontend engineering team
Companion document: `learnhub-api` 's Backend SRS (`SRS.md` in that repo) —
Section 7 of this document maps every screen to the endpoints it depends on there.

1. Introduction

1.1 Purpose

This document specifies what `learnhub-web` must build: every screen, the interactions and data each one needs, and the cross-cutting behavior (auth, routing, error handling) that ties them together. Visual design is specified separately (Stitch prototype); this document is about *requirements*, not layout.

1.2 Scope

LearnHub Cameroon is YouTube-shaped: courses are like videos (browse feed, watch page with comments), following a tutor is subscribing to a channel, liking is liking. Supporting a tutor financially (Mobile Money) is a separate "tip the creator" action, distinct from the like/comment/follow social layer, and does **not** gate access to course content.

1.3 Audience

Frontend developers and their lead implementing `learnhub-web`. Backend developers should read Section 7 only, for the API contract this side expects.

1.4 Definitions

See the Backend SRS Section 1.4 — the same role/entity definitions apply (Guest, Student, Tutor; Support/Subscription $\neq$ paywall).

2. System Overview

2.1 Tech stack

Vite + React 19, Tailwind CSS v4 (CSS-first, no `tailwind.config.js`), React Router 7, hosted on Vercel. `src/lib/api.js` is the single point of contact with the backend — no component calls `fetch` directly.

2.2 User roles / shells

- **Guest / Student** use the public shell: top navbar (logo, Courses, Tutors, About, Search, and Login/Signup or an Avatar menu depending on auth state).
- **Tutor**, once logged in, uses a distinct creator-studio shell: left sidebar (Overview, My Courses, Earnings, Profile, Settings) instead of the public navbar.
- Role determines the post-login landing route (Section 3) — this must be read from the JWT/user object returned at login, not assumed.

2.3 Known current gaps to fix as part of this build

- `apiFetch()` (`src/lib/api.js`) currently throws a generic `API request failed: ${status} ${statusText}` on any non-OK response instead of parsing the backend's `{ success: false, message: "..."` body. Fix this before wiring any form that needs to show a real error ("Invalid credentials", "Email already registered") — it's a prerequisite for FR-2 (Signup) and FR-3 (Login) below, not optional polish.
- All backend responses are one level deep: `const res = await apiFetch('/courses');` `setItems(res.data)`, never `setItems(res)`.

3. Site Map

Route	Screen	Access	Shell
<code>/</code>	Landing page	Public	Public navbar
<code>/courses</code>	Courses (browse/feed)	Public	Public navbar
<code>/tutors</code>	Tutors (browse)	Public	Public navbar
<code>/courses/:id</code>	Course detail	Public (gated actions inside)	Public navbar
<code>/tutors/:id</code>	Tutor profile	Public	Public navbar

Route	Screen	Access	Shell
/about	About	Public	Public navbar
/signup	Signup	Public	Minimal/centered
/login	Login	Public	Minimal/centered
/account	My Account / Profile	Student (auth)	Public navbar
/account/settings	Edit Profile / Settings	Student (auth)	Public navbar
/tutors/:id/support	Support/Subscribe to a tutor	Student (auth)	Public navbar
/account/subscriptions	My Subscriptions dashboard	Student (auth)	Public navbar
/dashboard	Tutor Dashboard (overview)	Tutor (auth)	Sidebar
/dashboard/courses	My Courses (manage)	Tutor (auth)	Sidebar
/dashboard/courses/new , /dashboard/courses/:id/edit	Create / Edit Course	Tutor (auth)	Sidebar
/dashboard/earnings	Earnings & Payouts	Tutor (auth)	Sidebar
/dashboard/profile	Tutor Public Profile Edit	Tutor (auth)	Sidebar
*	404	Public	Public navbar

Route guarding: any `/account/*` route requires a logged-in user; any `/dashboard/*` route requires `role: "tutor"`. A Student hitting a `/dashboard/*` URL should redirect to `/account`, and vice versa — role, not just auth state, must gate the tutor routes.

4. Functional Requirements by screen group

4.1 Public

- **Landing page** — hero + "Why LearnHub" + featured tutors row. Featured tutors can be a static/curated list initially if no "trending" endpoint exists yet — don't block this screen on backend ranking logic.
- **Courses (feed)** — paginated grid, filter by category, client-side or server-side sort. Must handle three states explicitly: loading (skeleton), empty (no results for filter), error (fetch failed — show retry, not a blank screen).
- **Tutors (browse)** — same three states as Courses. Filter by subject tag.
- **Course detail** — shows like count and comments to everyone; the *action* of liking/commenting is gated (Section 4.3). Must handle a course ID that doesn't exist (404, not a crash).
- **Tutor profile** — bio + their published courses grid; Follow button gated same as likes.
- **About / 404** — static content, no data dependency.

4.2 Auth

- **Signup** — name, email, password, role toggle (Student/Tutor). Client-side validation (required fields, email format, password min length) before hitting the API; server error message (once FR fix above lands) must surface inline per field or as a form-level banner, not a raw alert.
- **Login** — email, password. On success, store the JWT (see Section 5.1 on storage) and redirect based on role: Student → `/courses`, Tutor → `/dashboard`.

4.3 Community-gated actions (appear across Course detail / Tutor profile)

- If a Guest clicks Like, Comment, or Follow: do not silently fail — either disable the control with a tooltip ("Log in to like this course") or open a login prompt, then return the user to the same screen with the action completed post-login (see the auth-gate journey in the design brief).
- Once logged in, Like/Follow are optimistic-UI actions: update the count/state immediately, roll back if the request fails.
- Comment submission: disable the submit button while pending, clear the input on success, surface a specific error on failure (e.g. comment too long) rather than a generic one.

4.4 Student account

- **My Account / Profile** — tabs or sections for Following (tutor cards) and Liked Courses (course cards), both paginated if the list can grow large.
- **Edit Profile / Settings** — avatar upload, name, email, bio, password change (requires current password), save with clear success/error feedback.
- **Support/Subscribe to a tutor** — amount selection, Mobile Money provider (MTN/Orange), phone number input with format validation. On submit, show a pending state (the charge is confirmed async via backend webhook, not synchronously) — don't imply instant success before the backend confirms.
- **My Subscriptions dashboard** — list with status (active/pending/failed/cancelled), next billing date, and a cancel action with a confirmation step (this is a billing cancellation, treat it with the same care as any destructive/financial action).

4.5 Tutor dashboard

- **Overview** — summary cards (subscribers, earnings, views, rating) and a recent activity feed. Must have a distinct **empty state** for a brand-new tutor (0 subscribers, 0 courses) vs the populated state — don't design only for the populated case.
- **My Courses** — table with status, views, likes, comments, per-course earnings, and row actions (Edit/Unpublish/Delete). Delete must confirm before firing.
- **Create / Edit Course** — form with draft-save vs publish as two distinct actions; validate required fields (title, price, category) before allowing publish specifically (draft can be looser).
- **Earnings & Payouts** — this-month-vs-last comparison, active subscriber list, payout history. This screen's data depends on Phase 2 backend endpoints (see Backend SRS Section 7) — build the UI now, but gate it behind a feature flag or clearly-marked "coming soon" if the backend endpoints aren't live yet, rather than shipping a broken fetch.
- **Tutor Public Profile Edit** — banner/avatar upload, display name, subject tags, bio; this is what renders on the public Tutor profile screen, so preview the change against that same component if possible.

5. Cross-Cutting Requirements

5.1 Auth state

- Use a Context-based `AuthProvider` (per this track's Week 3 plan) wrapping the app, exposing `user`, `token`, `login()`, `logout()`, and `isAuthenticated`.
- **JWT storage:** do not store the token in `localStorage`. Prefer an `httpOnly` cookie set by the backend, or in-memory storage with a silent refresh strategy if the

backend adds refresh tokens (flagged as an open question in the Backend SRS). Confirm the chosen approach with backend before building `login()`, since it changes how `apiFetch` attaches the token.

- `apiFetch` must attach `Authorization: Bearer <token>` automatically for authenticated calls once the storage approach is decided — don't repeat this in every component.

5.2 Response handling

- Every `apiFetch` caller expects `res.data`, not `res` — see Section 2.3.
- Every form submission must handle three states: idle, pending (disable submit, show spinner), and error (show `res.message` once the backend returns it, per the FR-7 contract in the Backend SRS).

5.3 Role-based routing

- Implement as a `<ProtectedRoute role="student|tutor">` wrapper (or equivalent), not ad-hoc checks scattered inside individual page components.

6. Non-Functional Requirements

- **Responsive:** mobile-first; every screen in Section 3 must be usable at a small (~375px) width, not just desktop — the product's actual users will include mobile-first Mobile Money users.
- **Performance:** lazy-load route-level code (`React.lazy` + route-based splitting) for the tutor dashboard bundle, since Students never load it. Lazy-load below-the-fold images on Courses/Tutors grids.
- **Accessibility:** all interactive elements (Like, Follow, Support, form inputs) must be keyboard-reachable and have visible focus states; images need `alt` text; form errors must be associated with their field (not just color).
- **Browser support:** current evergreen Chrome/Firefox/Safari, plus mobile Safari/Chrome — no IE/legacy support needed.

7. API Dependency Matrix

Cross-reference with the Backend SRS Section 5 for full endpoint definitions.

Screen	Depends on
Courses feed	GET /api/courses
Course detail	GET /api/courses/:id , GET /api/courses/:id/comments , POST/DELETE .../likes , POST .../comments
Tutors browse	GET /api/tutors
Tutor profile	GET /api/tutors/:id , POST/DELETE .../follow
Signup / Login	POST /api/auth/register , POST /api/auth/login
My Account	GET /api/me , plus liked-courses/following lists (confirm exact list endpoints with backend — not yet itemized in Backend SRS Section 5, flag if missing)
Edit Profile	PATCH /api/me
Support a tutor	POST /api/subscriptions
My Subscriptions	GET /api/subscriptions/me , DELETE /api/subscriptions/:id
Tutor Dashboard overview	GET /api/tutors/me/earnings (Phase 2)
My Courses (manage)	GET/POST/PATCH/DELETE /api/courses (scoped to own)
Earnings & Payouts	GET /api/tutors/me/earnings , GET /api/tutors/me/payouts (Phase 2)
Tutor Profile Edit	PATCH /api/tutors/:id

Gap to raise with backend: My Account's "Following" and "Liked Courses" lists need dedicated list endpoints (e.g. GET /api/me/following , GET /api/me/likes) that aren't yet in the Backend SRS's endpoint table — confirm before building that screen.

8. Phasing

Phase	Scope
MVP (this cycle)	Landing, Courses, Tutors, Course detail, Tutor profile, About, 404, Signup, Login, community-gated actions
Phase 2	My Account, Edit Profile, Support/Subscribe, My Subscriptions, full Tutor Dashboard (Overview, My Courses, Create/Edit Course, Earnings & Payouts, Profile Edit)

9. Open Questions / Decisions Needed

1. JWT storage strategy — blocks `AuthProvider` implementation (Section 5.1).
2. Missing list endpoints for Following/Liked Courses (Section 7).
3. Whether Earnings & Payouts ships as a "coming soon" placeholder or is held back entirely until Phase 2 backend work lands.